

Video Link: [Focus_Buddy.MOV](#)

GitHub repository link: <https://github.com/MichaelAHeredia/Focus-Buddy.git>

Focus Buddy

Motivation

Focus Buddy sole purpose is to help people boost their productivity. Whether it's a university student studying for finals, a professional getting a speech done, or just somebody enjoying a book, Focus Buddy is there for you in your room or office. Born as a student project for IoT at University of Illinois Urbana-Champaign, the idea surged from a common problem we all faced, as many other college students and even people in their work environment: we tend to lose focus easily. This is a problem that has been accentuated by social media, as Mills stated, "Research has shown that over the past couple of decades people's attention spans have shrunk in measurable ways." (2023). As AI and LLM's keep on getting more intertwined in people's lives, these issues are likely to keep on increasing, some research even states that interrupted work can get some health-related consequences as "more stress, higher frustration, more time pressure and effort" (Mark et al., n.d.). Even with the existence of current productivity boosting methods, like Pomodoro Timers, do not disturb applications, and even advice from Google's former CEO "if you're trying to focus at work or relax: 'turn off your phone'" (Chandonnet, 2025). They are still not good enough for most people whether they depend on notifications in a work-related way, or just simply keep struggling to focus, mainly because there is not a physical aspect being measured, monitored and controlled where people get their work done. Thus, we decided to get hands in to action to try and help people get the most of their working time.

Technical approach

Regarding the technical approach, it all starts with our hardware and physical set up for the product. We decided to start by collecting physical data from our user's area, since that's our main material (literal features/inputs of our future model) to provide positive feedback that boosts productivity. Thus, we decided to use a microcontroller to do this since it's simple to set up sensor information collection, a basic diagram is shown on figure 2, where one temperature sensor represents the sound sensor, the one connected to the digital input and the red switch represents our IMU sensor, specifics of hardware are on the following section of this document. Once

the data is collected from the sensor inputs, it flows through our serial connection to our computing device (initially it was going to be a Raspberry Pi but ended up being one of our computers since it was easier to set up the CV setup there). Then, the computing device also gets information from the device camera that with CV algorithm detects if the person is focused (looking at the screen) or not. All this information gets sent into a csv in order to be used for our ML algorithm that intends to predict (the output) the best physical conditions for the users environment (specifics in the following section). Finally, we decided to sketch a phone holder design that was conceptualized on a notebook using calipers on an iPhone 15 Pro with a pretty thick case although the proportions and shape of other phone models were considered in its design.

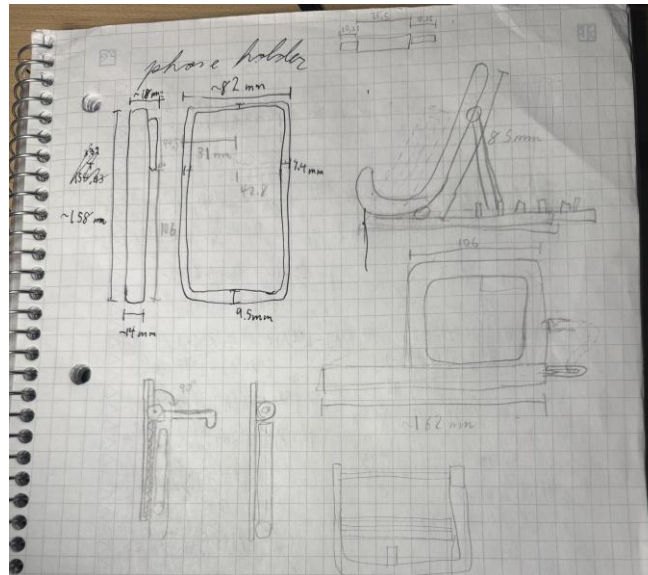


Figure 1: Initial phone holder sketch

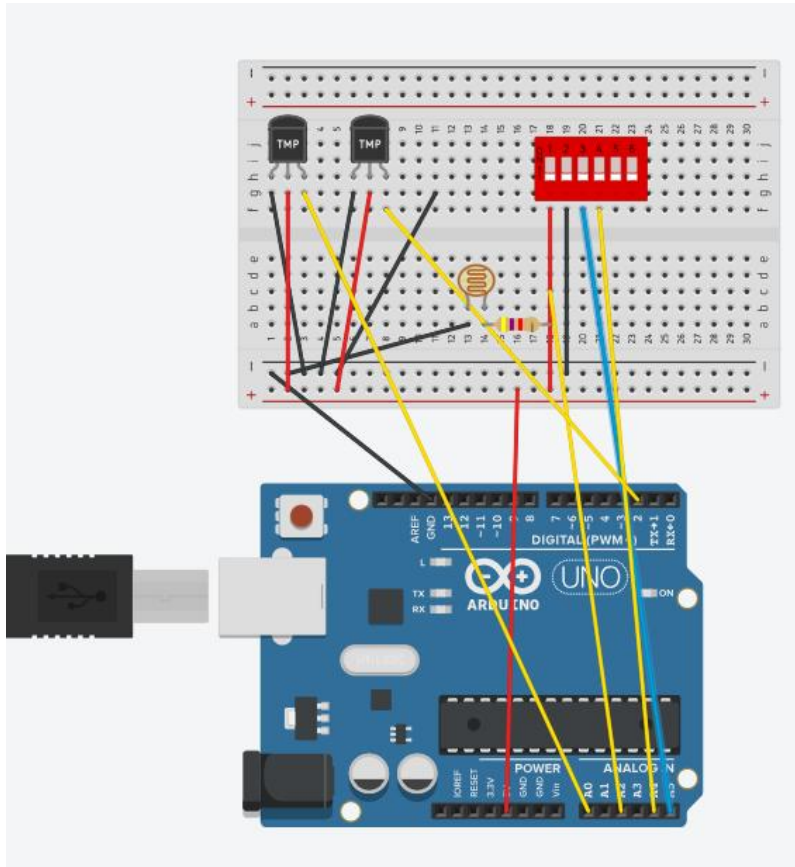


Figure 2: Initial hardware assembly sketch. Temperature, Sound, Light, and Movement sensors.

Implementation details

Hardware wise, we started with a microcontroller, specifically an Arduino UNO, attached to a wood platform with a breadboard. In the breadboard we arranged the sensors and their respective connections. Specifically, we used an IMU (LSM6DSO) to measure the users desktop movement, this was connected to analog inputs of the Arduino for both axis movement and accelerometer movement, the sensor itself didn't need any external resistance since it has one inside already. Then, we used an LDR Photoresistor sensor, that outputs analog signals to another port of our microcontroller, this one used a 4.7k Ohms resistance. Following, we connected our digital temperature sensor to the digital inputs of the Arduino, and lastly we connected the audio sensor to an analog input, on this last sensor we had to physically tune the sensitivity of the sensor (literally turn the potentiometer in it) in order to get accurate readings. You can see images of the process of assembly and final assemble product:

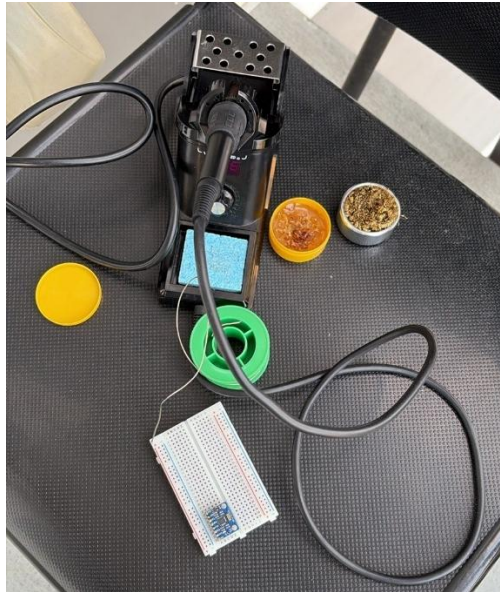


Figure 3: Soldering of IMU sensor pins

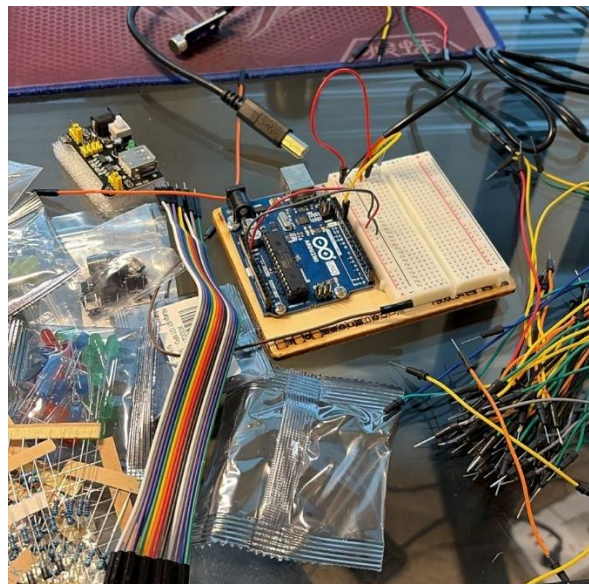


Figure 4: Hardware assembly process

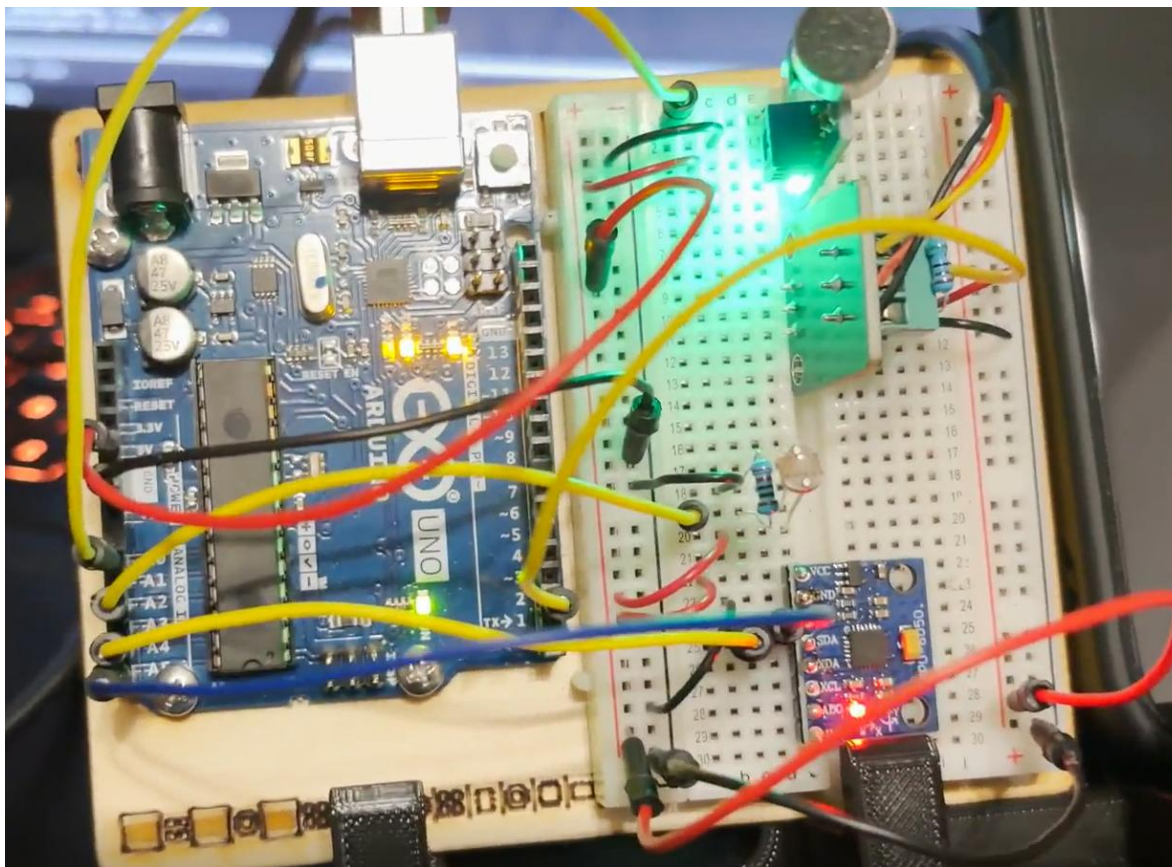


Figure 5: Final hardware assembled and working, outputs serial to computer

Regarding the camera, we used our computer camera for simplicity, and for the CV face/focus recognition we used the libraries CV2 by OpenCV, mediapipe by Google developers, and numpy.

Regarding the machine learning algorithm to predict best output, we decided to go with python for the main machine learning code for its ease of use and because Michael H. is very familiar with both it and C code so if lessening overhead was a problem in the future, he could reconvert the program from a virtual language to lower-level software. During initial testing, csv docs composed of fake data or even randomly generated data in the python code were used to train and test the model. After a while we decided to consolidate from several interacting python files to a single file to make possible implementation on a different OS easier. The final product outputs real time information of the best current environment, where the user tends to focus the most.

The notification listener was initially attempted with the python module winrt, but this was scrapped as Windows no longer supports it. It was then decided to create

a windows app (for direct supported integration) that would interface through a local http connection to a python receiver program.

Regarding the phone holder implementation, it was designed in Fusion 360 using the dimensions from the above-mentioned phone and estimated dimensions from the initial sketch. A phone model was made in fusion 360 to help with visualization.

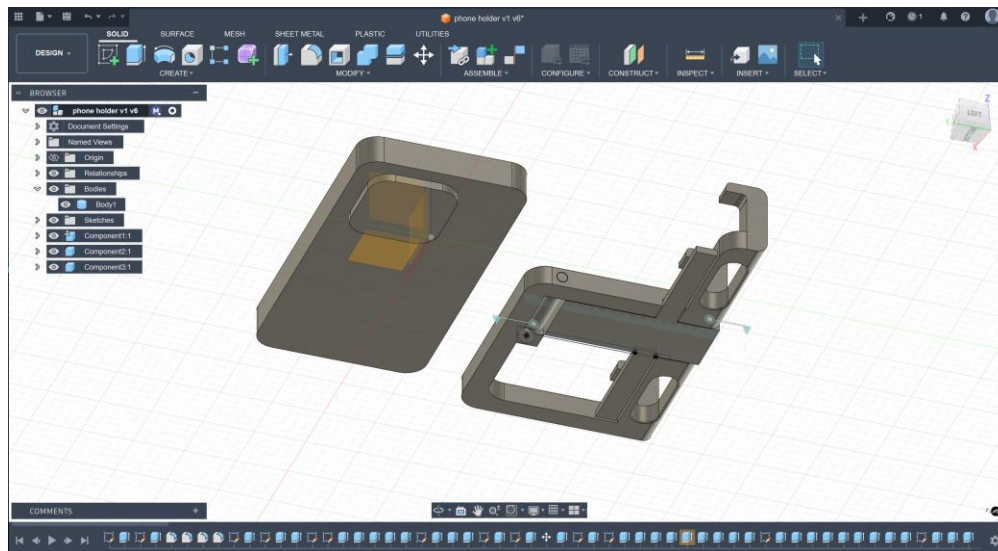


Figure 6: Phone holder 3D model

The print was then completed on an Ender 3 running standard specifications. Supports were selected but for some reason did not print. Thankfully, the print turned out to work fine anyways.

Results

In terms of the three main components of the project: phone control, algorithmic learning, and environmental sensing, all three were successful. We may not have developed as far as we hoped in some areas, but we developed more than we had hoped in others and saw proof of concept in all.

Machine learning algorithm:

Our algorithm was able to output recommended optimal conditions based on sensor data. At first with training from initially collected data, and, after some more development, while running live and collecting sensor data. In the future, this would be consolidated on a raspberry pi or custom PCB and signal different smart devices to

adjust temperature, lighting, etc. We also wanted to optimize the best ML algorithm to apply here but did not have the time to do this.

Sensor array:

The sensor array was able to successfully output data in csv format to the main python program both live and during training (training separately from collecting data mainly took place in the initial tests of the algorithm). Of the initially proposed sensors, we couldn't get a hold of the pressure sensor to use it on our project, however, all the other sensors worked fine after some fine tuning of the cables and connections, so we got some decently accurate results. It's important to note that in the case of the IMU and sound sensor, since their values were so volatile, we decided to make a threshold for our ML features for simplicity.

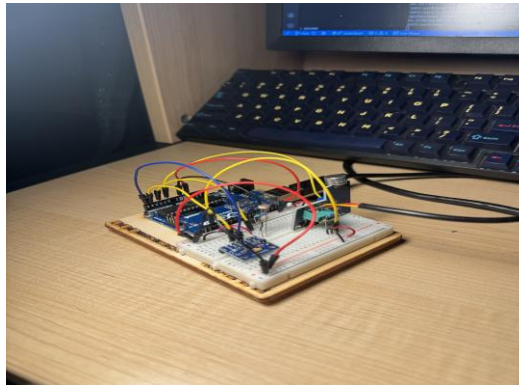


Figure 7: assembled hardware

Ideally, we would like to have the sensors implemented on a PCB that would also run the algorithm or perhaps be connected to a raspberry pi that would run the machine learning code. We had many ideas for configurations and placement of the hardware that could be implemented in the future such as attaching it to the chair or making a gyroscope wristband or neck attachment of some sort to detect movement during training the algorithm initially.

Phone control:

The phone holder itself was successful albeit a rough print. It is able to flatten for portability and is adjustable in angle as well as being quite sturdy.



Figure 8: finished phone holder back



Figure 9: finished phone holder folded

The notification listener windows app also worked out well. We were able to receive text messages on the phone and have them show up in the output of the python file when running alongside the windows app.

Our vision for the phone holder would be able to lock to different locations, have a slightly cleaner design, and possibly even lock one's phone as seen fit by the algorithm. The notification listener would ideally be a notification blocker and filter so that the phone could be silenced while the algorithm sorted out the important and not important information. So, things like meetings or classes would be both calculated in the ideal conditions part and be allowed to reach the user while social media updates would be blocked. This was something we had planned to implement but did not have the time to complete it.

Contribution

Each member contributed to some degree in both hardware, software and report writing. At first, we split up the workload into Michael H. developing an initial machine learning algorithm, Michael C. developing a focus detection algorithm for the camera module, and Nicolas developing hardware assembly and initial software to gleam data from the sensors. During the next group of weeks Michael C. and Nicolas worked together to join their work and develop a consolidated system to output full live data sets to the algorithm while Michael H. worked to make the algorithm easy to use, flushed out, and able to use different machine learning algorithms like random forest or ridge regression to visualize results differently. Finally, in the last few weeks, Nick and Michael C. finalized their work on the main hardware with fully functional sensors, and Michael C. consolidated the code to a single python file and outputted recommendations based on ambient information. Michael H. designed and printed the adjustable phone holder and created the windows app to listen to notifications on computers and phones.

References and libraries:

- Mills, K. (2023, February). Speaking of Psychology: Why our attention spans are shrinking, with Gloria Mark, PhD (G. Mark, Interviewer). American Psychological Association. <https://www.apa.org/news/podcasts/speaking-of-psychology/attention-spans>
- Mark, G., Department of Informatics, University of California, Irvine, & Institute of Psychology, Humboldt University, Berlin, Germany. (n.d.). The cost of interrupted work: more speed and stress. In *Department of Informatics, University of California, Irvine* [Journal-article]. <https://ics.uci.edu/~gmark/chi08-mark.pdf>
- Chandonnet, H. (2025, July 21). *Google's former CEO has some simple advice if you're trying to focus at work or relax: "Turn off your phone."* Business Insider. <https://www.businessinsider.com/eric-schmidt-advice-workers-focus-relaxation-turn-off-phone-2025-7>
- NumPy documentation — NumPy v2.3 Manual. (n.d.). <https://numpy.org/doc/stable/>
- mediapipe. (2025, February 6). PyPI. <https://pypi.org/project/mediapipe/>
- OpenCV: OpenCV modules. (n.d.). <https://docs.opencv.org/4.x/>